LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 2

Stack dan Queue

Oleh:

Nadya Nadhir Soraya NIM. 2510817120022

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 2

Laporan Praktikum Algoritma & Struktur Data Modul 2: Stack dan Queue ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Nadya Nadhir Soraya
NIM : 2510817120022

Menyetujui, Asisten Praktikum	Mengetahui, Dosen Penanggung Jawab Praktikum
Muhammad Fauzan Ahsani NIM. 2310817310009	Ir. Muhamamd Alkaff, S,Kom, M.Kom, Ph.D NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN ii
DAFTAR ISI iii
DAFTAR GAMBAR iv
DAFTAR TABEL v
SOAL 1 1
A. Source Code 1
B. Pembahasan 3
SOAL 2 5
A. Source Code 6
B. Output Program 8
C. Pembahasan 8
SOAL 3 11
A. Source Code 11
B. Pembahasan 13
SOAL 4 16
A. Source Code 17
B. Output Program 18
C. Pembahasan 18
TAUTAN GIT 21

DAFTAR GAMBAR

Gambar 1 Screenshot Output Program Prob1.cpp..... 8
Gambar 2 Screenshoot Output Program prob2.cpp..... 18

DAFTAR TABEL

Tabel 1 Source Code Stack.cpp 1
Tabel 2 Source Code Prob1.cpp 6
Tabel 3 Source Code queue.cpp ..... **Error! Bookmark not defined.**
Tabel 4 Source Code prob2.cpp 17

SOAL 1

Pada file stack.h yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file stack.cpp. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan throw.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

Tabel 1 Source Code Stack.cpp

```
#include "stack.h"
#include <stdexcept>

void init(Stack* s) {
    s->top = nullptr;
}

bool isEmpty(const Stack* s) {
    return s->top == nullptr;
}

bool isFull(const Stack* s) {
    return s->top == s->data + MAX - 1;
}

void push(Stack* s, int value) {
    if (isFull(s)) {
```

```
        throw std::overflow_error("Error: Stack penuh (Overflow)!");
    }

    if (isEmpty(s)) {
        s->top = s->data;
    } else {
        s->top++;
    }

    *(s->top) = value;
}

void pop(Stack* s) {
    if (isEmpty(s)) {
        throw std::underflow_error("Error: Stack kosong, tidak bisa pop (Underflow)!");
    }

    if (s->top == s->data) {
        s->top = nullptr;
    } else {
        s->top--;
    }
}

int peek(const Stack* s) {
    if (isEmpty(s)) {
        throw std::underflow_error("Error: Stack kosong, tidak ada elemen teratas!");
```

```
45      }
46      return *(s->top);
47  }
```

B. Pembahasan

Implementasi Stack

Pada soal 1, saya membayangkan Stack sebagai tumpukan piring. Pointer `top` adalah jari telunjuk kita yang hanya fokus melihat ke piring paling atas.

1. Fungsi `init(Stack* s)`
   - Alur : Pada fungsi init baris ke [5] mengatur `top` menjadi `nullptr` (tidak menunjuk apa-apa).
   - Pembuktian: Langkah ini wajib agar sistem tahu bahwa meja tumpukan masih kosong. Tanpa inisialisasi ini, sistem bisa salah membaca data acak (*garbage value*) dari sisa memori lain.
   - Kompleksitas: Waktu O(1) (hanya satu instruksi), Ruang O(1).
2. Fungsi `isEmpty(const Stack* s)` dan `isFull(const Stack* s)`
   - Alur: `isEmpty` mengecek apakah `top` menunjuk ke tempat kosong. `isFull` mengecek apakah top sudah mentok di batas paling akhir kapasitas array `MAX`.
   - Pembuktian: Ini adalah gerbang keamanan program. `isEmpty` mencegah program mengambil piring yang tidak ada (mencegah *crash*), dan `isFull` mencegah program memaksakan piring masuk saat kapasitas penuh (mencegah *overflow* memori).
   - Kompleksitas: Waktu O(1) (hanya perbandingan logika), Ruang O(1).
3. Fungsi `push(Stack* s, int value)`
   - Alur: 1. `Cek isFull`. Jika penuh, hentikan program dengan pesan `overflow_error`.
     2. Jika aman, naikkan `top` (jari telunjuk) satu langkah ke ruang kosong di atasnya `s->top++`.
     3. Taruh piring baru tepat di ruang kosong tersebut.

- Pembuktian: Dengan selalu menaikkan top ke atas setiap ada data baru, kita menjamin berjalannya sifat LIFO (*Last In, First Out*) secara matematis. Data terakhir pasti menjadi yang teratas.
- Kompleksitas: Waktu O(1) (tidak ada proses pengulangan/ *looping*), Ruang O(1) (tidak membuat array baru).

4. Fungsi `pop(Stack* s)`
   - Alur: 1. `Cek isEmpty`. Jika kosong, tolak dengan pesan `underflow_error`.

     2. Jika ada isinya, kita cukup menurunkan `top` (jari telunjuk) ke bawah satu tingkat `s->top--`.
   - Pembuktian: Kita tidak perlu menghancurkan atau menghapus data secara fisik dari memori komputer. Cukup dengan menurunkan `top`, piring paling atas tadi secara logis sudah "diabaikan" dan diuntungkan karena sangat menghemat daya kerja komputer.
   - Kompleksitas: Waktu O(1) (hanya pergeseran satu alamat memori), Ruang O(1).
5. Fungsi `peek(const Stack* s)`
   - Alur: Selama tumpukan tidak kosong, kembalikan nilai yang tepat yaitu nilai yang sedang ditunjuk oleh `top` saat ini `*(s->top)`.
   - Pembuktian: Operasi ini bersifat *read-only*. Kita hanya "mengintip" piring teratas untuk bahan perhitungan, tanpa mengambilnya atau mengubah posisi top sama sekali.
   - Kompleksitas: Waktu O(1), Ruang O(1).

SOAL 2

Time Limit: 1.0 s
Memory Limit: 64 MB

Si Palui menemukan sebuah mesin hitung tua di gudang. Mesin tersebut memiliki cara kerja yang tidak biasa. Alih-alih menggunakan format perhitungan seperti yang biasa kita kenal, mesin ini membaca input berupa deretan simbol yang dipisahkan oleh spasi. Setiap simbol bisa berupa:

• sebuah bilangan bulat, atau

• sebuah operator (+, -, *, /)

Cara kerja mesin adalah sebagai berikut:

• Mesin membaca simbol dari kiri ke kanan.

• Jika simbol adalah bilangan, maka bilangan tersebut disimpan ke dalam memori mesin.

• Jika simbol adalah operator, maka mesin akan mengambil dua nilai yang paling baru dimasukkan ke dalam memori yang disimpan, melakukan operasi sesuai operator tersebut, lalu menyimpan kembali hasilnya ke dalam memori.

Setelah seluruh simbol diproses, akan tersisa satu nilai di dalam memori. Nilai inilah yang ditampilkan sebagai hasil akhir.

Sebagai asisten Si Palui, tugas Anda adalah membantu menghitung hasil dari mesin tersebut.

Batasan

• $1 \leq n \leq 100$

• $-1000 \leq Ai \leq 1000$

Keterangan:

• n adalah jumlah simbol dalam ekspresi

• Ai adalah nilai bilangan pada input

• Simbol yang valid hanya:

‣ bilangan bulat

‣ operator: +, -, *, /

Format Input

N

A1 A2 ... An

Keterangan:

• Baris pertama n menyatakan jumlah atau panjang simbol

• Baris kedua berisi simbol A sebanyak n buah.

Format Output Sebuah bilangan hasil dari kalkulator Si Palui.

Contoh Kasus

Input	**Output**
3 2 3 +	5
9 5 1 2 + 4 * + 3 -	14
5 10 2 / 3 *	15

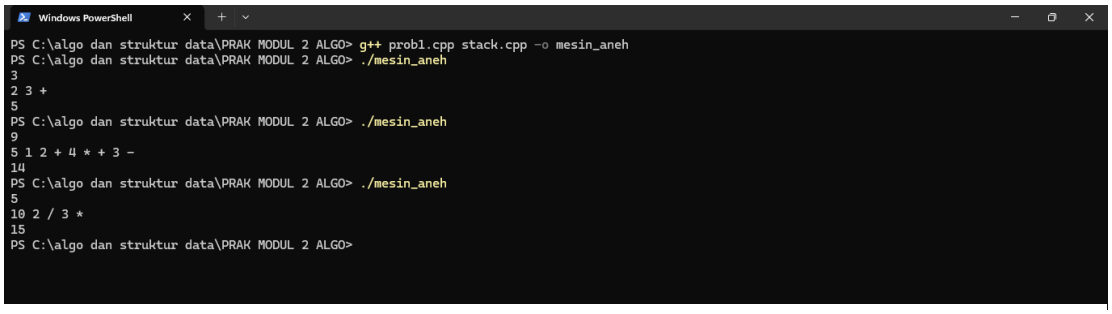
A. Source Code

Tabel 2 Source Code Prob1.cpp

```
#include <iostream>
#include <string>
#include <stdexcept>
#include "stack.h"

using namespace std;

int main() {
    int n;
    if (!(cin >> n)) return 0;

    Stack s;
```

```
    init(&s);

    for (int i = 0; i < n; i++) {
        string simbol;
        cin >> simbol;

        if (simbol == "+" || simbol == "-" || simbol == "*" || simbol == "/") {
            int b = peek(&s); pop(&s);
            int a = peek(&s); pop(&s);

            if (simbol == "+") push(&s, a + b);
            else if (simbol == "-") push(&s, a - b);
            else if (simbol == "*") push(&s, a * b);
            else if (simbol == "/") push(&s, a / b);
        } else {
            push(&s, stoi(simbol));
        }
    }

    cout << peek(&s) << endl;

    return 0;
}
```

B. Output Program



Gambar 1 Screenshot Output Program Prob1.cpp

C. Pembahasan

Pembahasan Source Code Prob1.cpp

Program pada prob1.cpp adalah program utama yang menjalankan algoritma. Kita bisa membayangkan program ini seperti orang yang sedang membaca instruksi matematika dari kiri ke kanan, dan menggunakan tumpukan piring (*Stack*) sebagai alat bantu kotretannya.

1. Alur Algoritma (Logika Program)

- Persiapan: Program melihat terlebih dahulu berapa banyak simbol instruksi yang akan dibaca (`n`). Meja tumpukan kemudian dibersihkan dan disiapkan agar hitungan dimulai dari keadaan benar-benar kosong.
- Proses Membaca Instruksi: Program mulai membaca simbol satu per satu dari ujung kiri ke ujung kanan.
    - Jika membaca Angka: Angka tersebut langsung ditaruh sebagai "piring" baru paling atas pada tumpukan piring sebelumnya.
    - Jika membaca Operator (+, -, *, /): Program langsung mengambil dua piring teratas dari tumpukan. Piring pertama yang diangkat akan menjadi Angka Kanan, dan piring kedua di bawahnya menjadi Angka Kiri. Keduanya lalu dihitung sesuai operator tersebut, dan hasil akhirnya ditaruh kembali ke atas tumpukan.
- Hasil Akhir: Karena setiap dua angka selalu dilebur menjadi satu hasil, maka pada akhir instruksi pasti hanya akan tersisa tepat satu piring di atas meja. Piring terakhir itulah yang dicetak ke layar sebagai jawaban akhir.

2. Pembuktian Algoritma (Uji Kasus: 9 simbol input 5 1 2 + 4 * + 3 -)

Angka 9 di awal menandakan mesin akan membaca tepat 9 simbol secara berurutan. Aturan utamanya: Angka langsung ditumpuk, tapi jika ketemu operator, ambil 2 angka teratas (yang diangkat pertama jadi Kanan/`b`, yang kedua jadi Kiri/`a`).

- Menumpuk Angka: Pertama, mesin membaca angka 5, 1, lalu 2. Semuanya langsung ditumpuk. Posisi dari bawah ke atas menjadi: [5, 1, 2].
- Ketemu operator +: Mesin mengambil angka 2 (Kanan) dan angka 1 (Kiri). Karena operator pertambahan maka dua angka tersebut dijumlahkan. Hitung 1 + 2 = 3. Hasilnya ditumpuk kembali. Isi tumpukan sekarang: [5, 3].
- Baca bilangan bulat 4 lalu bertemu operator `*` (Kali): Mesin menumpuk angka 4, lalu mendeteksi tanda kali. Mesin mengambil angka 4 (Kanan) dan angka 3 (Kiri). Hitung 3 x 4 = 12. Ditumpuk kembali sehingga tumpukan menjadi: [5, 12].
- Ketemu operator +: Mesin mengambil 12 (Kanan) dan 5 (Kiri). Hitung 5 + 12 = 17. Tumpukan kini hanya berisi: [17].
- Baca bilangan bulat 3 lalu bertemu operator − : Mesin menumpuk 3. Saat membaca tanda kurang, mesin mengambil 3 (Kanan) lalu 17 (Kiri). Hitung 17 - 3 = 14. Hasil akhir ini ditumpuk, menyisakan: [14].

3. Analisis Kompleksitas

- Kompleksitas Waktu: O(n) Program ini beroperasi secara *linear*. Artinya, waktu yang dibutuhkan hanya bergantung pada seberapa banyak instruksi yang diberikan (n). Karena program menyapu instruksi dari kiri ke kanan menggunakan satu siklus perulangan, dan semua operasi tumpukan (taruh/ambil piring) berjalan secara instan atau O(1), maka total waktu komputasinya adalah murni O(n).

- Kompleksitas Ruang: O(n)

  Program ini membutuhkan memori tambahan berupa *Stack* (tumpukan). Pada skenario terburuk (*worst-case scenario*), yaitu ketika pengguna memasukkan deretan angka yang sangat panjang tanpa ada satupun tanda operator matematika, maka tumpukan tidak akan pernah menyusut. Akibatnya, tumpukan tersebut harus menampung seluruh angka sebanyak n buah secara bersamaan di dalam memori, sehingga batas memori ruangnya adalah O(n).

SOAL 3

Pada file queue.h yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file queue.cpp. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan throw.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

Tabel 3 Source Code queue.cpp

```
#include "queue.h"
#include <stdexcept>

void init(Queue* q) {
    q->front = nullptr;
    q->rear = nullptr;
}

bool isEmpty(const Queue* q) {
    return q->front == nullptr;
}

bool isFull(const Queue* q) {
    if (isEmpty(q)) return false;

    const int* next_rear = q->data + ((q->rear - q->data + 1) % MAX);
```

```

    return next_rear == q->front;
}

void enqueue(Queue* q, int value) {
    if (isFull(q)) throw std::overflow_error("Queue is full");

    if (isEmpty(q)) {
        q->front = q->data;
        q->rear = q->data;
    } else {
        q->rear = q->data + ((q->rear - q->data + 1) % MAX);
    }
    *(q->rear) = value;
}

void dequeue(Queue* q) {
    if (isEmpty(q)) throw std::underflow_error("Queue is empty");

    if (q->front == q->rear) {
        q->front = nullptr;
        q->rear = nullptr;
    } else {
        q->front = q->data + ((q->front - q->data + 1) % MAX);
    }
}
```

```

int front(const Queue* q) {
    if (isEmpty(q)) throw std::underflow_error("Queue is empty");
    return *(q->front);
}

int back(const Queue* q) {
    if (isEmpty(q)) throw std::underflow_error("Queue is empty");
    return *(q->rear);
}
```

B. Pembahasan

Implementasi Queue

Pada struktur data *Queue*, prinsip utama yang dijaga adalah FIFO (*First In, First Out*). Siapa yang masuk antrian lebih dulu, dia yang dilayani lebih dulu. Bayangkan ini seperti barisan kursi loket yang disusun melingkar (*Circular Queue*) dengan dua penunjuk arah: `front` (ujung depan, bertugas melayani) dan `rear` (ujung belakang, menunjuk orang paling terakhir masuk).

1. Fungsi `init(Queue* q)`

- Alur Algoritma: Mengatu `front` dan `rear` menjadi `nullptr`. Ibaratnya, kita memerintahkan kedua petugas untuk istirahat karena belum ada satu orang pun yang mengantri.
- Pembuktian: Langkah inisialisasi ini wajib. Jika tidak diatur menjadi *null*, penunjuk bisa saja mengarah ke *garbage value* di sisa memori komputer, yang membuat sistem salah mengira sudah ada antrian gaib yang sedang menunggu.
- Kompleksitas: Waktu O(1) (hanya pemberian nilai), Ruang O(1).

2. Fungsi `isEmpty(const Queue* q)` dan `isFull(const Queue* q)`

- Alur Algoritma: `isEmpty` mengecek apakah petugas depan tidak menunjuk siapa-siapa `nullptr`. Sementara `isFull` mengecek apakah kursi tepat di depan petugas belakang yaitu `rear` sudah diduduki petugas depan yakni `front`, yang berarti putaran kursi sudah terisi penuh.
- Pembuktian: Keduanya bertindak sebagai sistem pendeteksi keamanan murni. `isEmpty` mencegah program melayani antrian kosong (menghindari *crash underflow*), dan `isFull` mencegah orang baru menerobos masuk saat kursi sudah habis (menghindari *crash overflow*).
- Kompleksitas: Waktu O(1) (hanya membandingkan kondisi alamat memori), Ruang O(1).

3. Fungsi `enqueue(Queue* q, int value)`

- Alur Algoritma: Cek `isFull` terlebih dulu. Jika kosong total, arahkan `front` dan `rear` ke kursi pertama sekaligus. Jika sudah ada antrian, geser `rear` ke kursi selanjutnya. Karena kursinya melingkar (menggunakan rumus modulo `% MAX`), petugas akan otomatis memutar kembali ke kursi awal jika sudah sampai ujung.
- Pembuktian: Menjamin sifat FIFO dan memaksimalkan memori. Posisi antrian lama tidak terganggu, dan kursi kosong di depan tidak mubazir dibiarkan kosong.
- Kompleksitas: Waktu O(1) (tidak ada perulangan memindahkan data), Ruang O(1).

4. Fungsi `dequeue(Queue* q)`

- Alur Algoritma: Jika antrian sisa satu orang (`front == rear`), orang itu keluar lalu `front` dan `rear` direset ke `nullptr`. Jika lebih dari satu, cukup geser `front` ke orang berikutnya dengan cara melingkar yang sama `% MAX`.
- Pembuktian: Kita tidak perlu menghapus data dari memori atau memajukan seluruh barisan karena akan sangat lambat. Cukup menggeser penunjuk `front`, secara logika orang sebelumnya sudah dianggap "keluar".
- Kompleksitas: Waktu O(1) (hanya menggeser posisi *pointer*), Ruang O(1).

5. Fungsi `front(const Queue* q)` dan `back(const Queue* q)`

- Alur Algoritma: Pastikan antrian tidak kosong. Fungsi `front` mengembalikan nilai yang ditunjuk oleh `q->front`, sedangkan `back` mengembalikan nilai dari `q->rear`.
- Pembuktian: Operasi ini bersifat *read-only*. Sekadar "mengintip" siapa antrian terdepan dan terbelakang tanpa mengeluarkan mereka dari barisan.
- Kompleksitas: Waktu O(1), Ruang O(1).

SOAL 4

Si Palui memiliki sebuah deretan angka yang tersusun rapi. Ia tertarik untuk mengamati sebagian angka secara berurutan dengan jumlah tertentu.

Ia memilih sejumlah k angka yang bersebelahan, lalu menghitung jumlahnya. Setelah itu, ia menggeser pilihannya satu langkah ke kanan dan kembali menghitung jumlah dari angka-angka yang terpilih.

Proses ini dilakukan terus hingga tidak memungkinkan lagi untuk memilih k angka berurutan. Sebagai asistennya, bantulah Si Palui untuk menentukan hasil perhitungan tersebut.

Batasan

- $1 \leq n \leq 100$
- $-1 \leq k \leq n$
- $-1000 \leq Ai \leq 1000$

Keterangan:

- n: jumlah elemen
- k: ukuran jendela
- Ai: elemen array

Format Input

n k

A1 A2 ... An

Keterangan:

- Baris pertama n menyatakan jumlah atau panjang array. Lalu k besar atau jumlah elemen dalam satu kelompok ayng dibentuk
- Baris kedua berisi simbol A sebanyak n buah

Format Output

Cetak hasil tiap masing-masing kelompok dalam satu baris dipisahkan dengan spasi

Contoh Kasus

Input	Output
5 3 1 2 3 4 5	6 9 12
5 5 1 2 3 4 5	15
6 2 4 -1 2 -3 5 1	3 1 -1 2 6

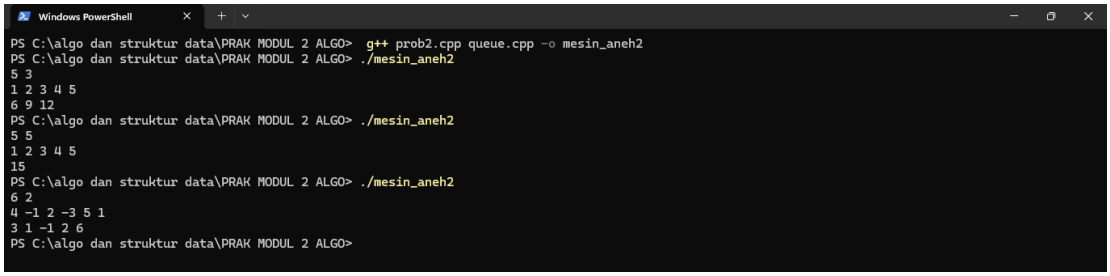
A. Source Code

Tabel 4 Source Code prob2.cpp

```
#include <iostream>
#include "queue.h"

using namespace std;

int main() {
    int n, k;
    if (!(cin >> n >> k)) return 0;
    if (k < 1) return 0;

    Queue q;
    init(&q);

    int sum = 0;

    for (int i = 0; i < n; i++) {
        int x;
        cin >> x;

        enqueue(&q, x);
        sum += x;

```

```
        if (i >= k) {
            sum -= front(&q);
            dequeue(&q);
        }

        if (i >= k - 1) {
            if (i > k - 1) cout << " ";
            cout << sum;
        }
    }
    cout << endl;

    return 0;
}
```

B. Output Program



Gambar 2 Screenshoot Output Program prob2.cpp

C. Pembahasan

Pembahasan Source Code prob2.cpp

Program ini menggunakan struktur data *Queue* berprinsip FIFO (*First In, First Out*), ibarat antrian loket dengan batas maksimal `k` kursi. Logikanya dikendalikan oleh dua penunjuk utama: `front` (ujung depan antrian yang bersiap melayani) dan `rear` (ujung belakang tempat pendatang baru masuk). Selama proses antrian bergeser,

sebuah variabel `sum` bertugas mencatat total nilai dari seluruh elemen yang saat ini sedang berada di dalam antrian tersebut.

1. Alur Algoritma

- Persiapan: Petugas melihat berapa total orang yang akan datang `n` dan berapa batas kursi yang tersedia `k`. Barisan kursi *Queue* disiapkan dalam keadaan kosong, dan catatan total nilai `sum` dimulai dari angka 0.
- Proses Masuk Antrian: Program memanggil orang satu per satu. Setiap orang baru akan diarahkan masuk ke barisan belakang (`enqueue` di posisi `rear`), dan nilai yang ia bawa langsung ditambahkan ke catatan petugas `sum`.
- Menjaga Batas Kursi: Ini adalah kunci utama algoritmanya. Jika jumlah orang yang masuk sudah melebihi batas kursi `k`, maka orang yang berada di ujung `front` (paling lama mengantri) harus segera dilayani dan dikeluarkan dari barisan `dequeue`. Nilai orang yang keluar tersebut langsung dikurangkan dari buku catatan sum.
- Mencetak Hasil: Setiap kali barisan sudah bergeser (atau orang di dalam barisan sudah mencapai target kelompok `k`), petugas akan mencetak angka total yang ada di catatannya ke layar.

2. Pembuktian Algoritma (Uji Kasus: n = 5, k = 3 dengan input 1 2 3 4 5) Angka 5 menandakan total data yang diproses, sedangkan 3 adalah batas maksimal kursi loket (*Queue*).

- Mengisi Kapasitas Awal: Tiga orang pertama bernilai 1, 2, dan 3 masuk berurutan ke posisi `rear`. Petugas menambahkan nilainya ke dalam catatan `sum` = 1 + 2 + 3 = 6. Karena batas 3 kursi sudah tercapai, hasil pertama dicetak: 6. Posisi antrian saat ini: [1, 2, 3].
- Pergeseran Pertama (Kursi Penuh): Orang bernilai 4 masuk ke *`rear`*, menambah total `sum` = 6 + 4 = 10. Karena kapasitas berlebih, orang yang paling lama duduk di `front` bernilai 1 dikeluarkan `dequeue`. Nilainya dikurangkan

dari catatan `sum` = 10 - 1 = 9. Hasil kedua dicetak = 9. Posisi antrian bergeser menjadi: [2, 3, 4].

- Pergeseran Kedua: Orang terakhir bernilai 5 masuk ke `rear`, menambah total `sum` = 9 + 5 = 14. Orang terlama di `front` bernilai 2 dikeluarkan. Nilainya dikurangkan dari catatan `sum` = 14 - 2 = 12. Hasil akhir dicetak = 12. Posisi antrian menjadi: [3, 4, 5].

3. Analisis Kompleksitas

- Kompleksitas Waktu: O(n) Program ini beroperasi secara linear. Waktu kerjanya murni bergantung pada jumlah total orang yang datang `n`. Seluruh proses menggeser antrian (memasukkan ke `rear` dan mengeluarkan dari `front`) dieksekusi secara instan (O(1)) tanpa adanya perulangan bersarang. Oleh karena itu, seberapa banyak pun datanya, waktu prosesnya adalah O(n).

- Kompleksitas Ruang: Program ini membutuhkan memori tambahan berupa antrian (Queue). Berkat kedisiplinan algoritma, barisan kursi ini secara praktis menjaga kapasitasnya di batas `k`. Meskipun saat orang baru masuk antrian sempat berisi `k + 1` orang, elemen terdepan pasti langsung dibuang saat itu juga untuk menjaga batas kelompok. Sehingga, beban memori maksimal yang digunakan program berbanding lurus dengan nilai `k`, yaitu O(k). (Catatan: Logika ini berjalan aman dengan syarat nilai k tidak melebihi kapasitas total MAX pada sistem Circular Queue).

TAUTAN GIT

https://github.com/DSA26-ULM/module-2-stack-and-queue-userdyaaa